

Pamela Duran

CIS 345

Final Project

Githublink: <https://github.com/Azaza999/Final-Project.git>

1. Project Overview

The objective of this project was to modify the default xv6 operating system kernel by replacing its standard Round-Robin CPU scheduler with a randomized, proportional-share Lottery Scheduler. To demonstrate this functionality, user-space system calls were implemented to assign CPU tickets to processes and retrieve system statistics, allowing for a verifiable 3:2:1 proportional execution ratio.

2. Implementation Details

Random Number Generation (LCG)

Because the standard xv6 kernel lacks a native random number generator, a Linear Congruential Generator (LCG) was implemented directly within kernel/proc.c. This algorithm provides the pseudo-randomness required to fairly select a winning ticket during each scheduling cycle, mimicking a randomized lottery.

The Two-Pass Scheduler Algorithm

The core of the modification resides in the scheduler() function inside kernel/proc.c. The default Round-Robin loop was replaced with a two-pass logic sequence:

1. **Pass One (Total Tickets):** The scheduler iterates through the process table (proc[]) to calculate the total number of tickets currently held by all RUNNABLE processes.
2. **Pass Two (Winning Process):** A random winning ticket is selected (random() % total_tickets). The scheduler then iterates through the process table a second time, accumulating a running sum of tickets until the sum exceeds the winning ticket number. The process that pushes the sum over the threshold is selected to run.

System Calls and Process Control

To facilitate user interaction with the scheduler, two new system calls were added:

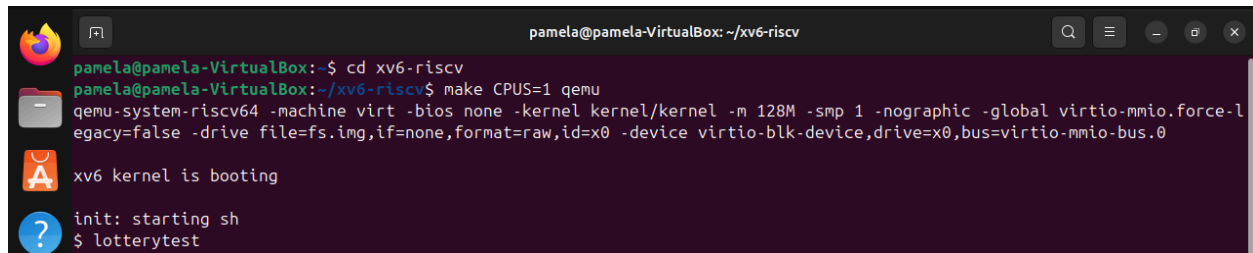
- sys_settickets: Allows a process to request a specific number of lottery tickets.

- `sys_getpinfo`: Retrieves statistical data (including CPU ticks and ticket counts) by populating a custom `pstat` structure. Additionally, the `allocproc()` function was modified to ensure that newly forked child processes inherit the exact ticket count of their parent process, rather than defaulting to 1.

3. Methodology and Testing

To verify the scheduler's behavior, a user-space test program (`lotterytest.c`) was written. This program forks three distinct child processes (`PID_A`, `PID_B`, and `PID_C`) and assigns them tickets in a **30:20:10** ratio.

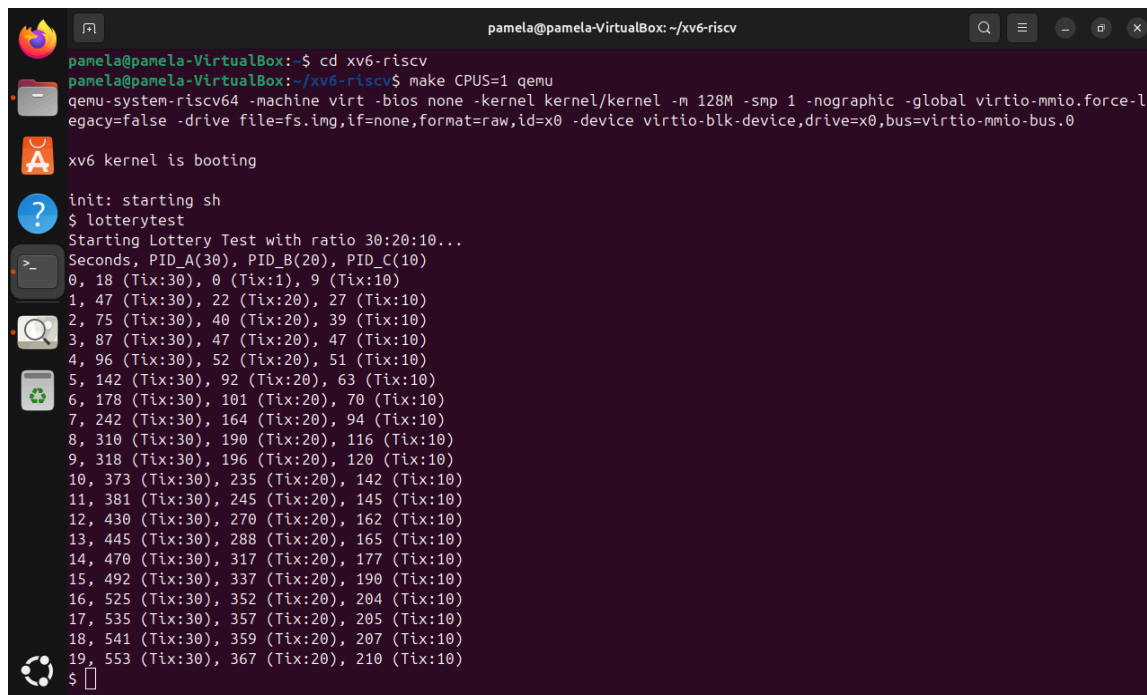
Testing Environment Constraint: By default, the QEMU emulator boots `xv6` with multiple hardware threads (harts). To properly test the scheduling algorithm, a CPU bottleneck had to be artificially created. The kernel was compiled and executed using the command `make CPUS=1 qemu`, forcing all three processes to compete for a single processor.



```
pamela@pamela-VirtualBox: ~/xv6-riscv
pamela@pamela-VirtualBox:~$ cd xv6-riscv
pamela@pamela-VirtualBox:~/xv6-riscv$ make CPUS=1 qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
init: starting sh
$ lotterytest
```

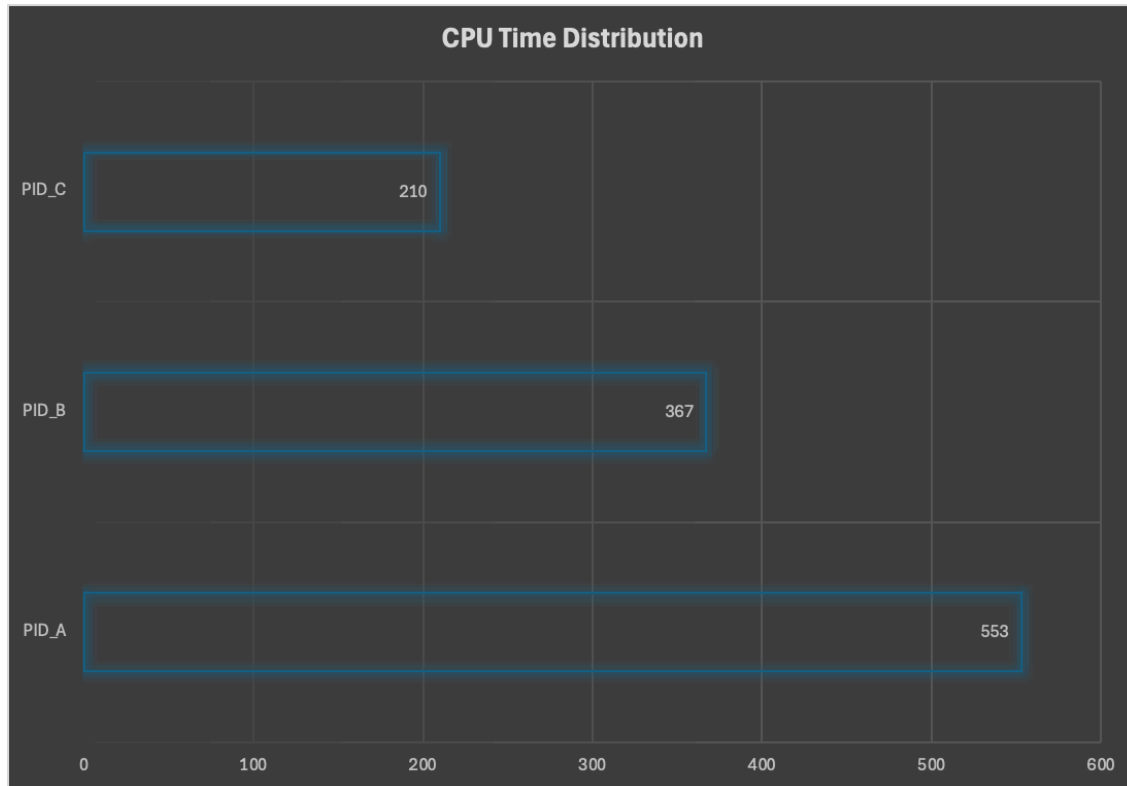
4. Results and Analysis

The `lotterytest` program was allowed to run for 20 seconds, continuously querying the kernel via `getpinfo()` to track how many CPU ticks were awarded to each process.



```
pamela@pamela-VirtualBox:~/xv6-riscv
pamela@pamela-VirtualBox:~$ cd xv6-riscv
pamela@pamela-VirtualBox:~/xv6-riscv$ make CPUS=1 qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
xv6 kernel is booting
init: starting sh
$ lotterytest
Starting Lottery Test with ratio 30:20:10...
Seconds, PID_A(30), PID_B(20), PID_C(10)
0, 18 (Tix:30), 0 (Tix:1), 9 (Tix:10)
1, 47 (Tix:30), 22 (Tix:20), 27 (Tix:10)
2, 75 (Tix:30), 40 (Tix:20), 39 (Tix:10)
3, 87 (Tix:30), 47 (Tix:20), 47 (Tix:10)
4, 96 (Tix:30), 52 (Tix:20), 51 (Tix:10)
5, 142 (Tix:30), 92 (Tix:20), 63 (Tix:10)
6, 178 (Tix:30), 101 (Tix:20), 70 (Tix:10)
7, 242 (Tix:30), 164 (Tix:20), 94 (Tix:10)
8, 310 (Tix:30), 190 (Tix:20), 116 (Tix:10)
9, 318 (Tix:30), 196 (Tix:20), 120 (Tix:10)
10, 373 (Tix:30), 235 (Tix:20), 142 (Tix:10)
11, 381 (Tix:30), 245 (Tix:20), 145 (Tix:10)
12, 430 (Tix:30), 270 (Tix:20), 162 (Tix:10)
13, 445 (Tix:30), 288 (Tix:20), 165 (Tix:10)
14, 470 (Tix:30), 317 (Tix:20), 177 (Tix:10)
15, 492 (Tix:30), 337 (Tix:20), 190 (Tix:10)
16, 525 (Tix:30), 352 (Tix:20), 204 (Tix:10)
17, 535 (Tix:30), 357 (Tix:20), 205 (Tix:10)
18, 541 (Tix:30), 359 (Tix:20), 207 (Tix:10)
19, 553 (Tix:30), 367 (Tix:20), 210 (Tix:10)
$
```

As shown in the graph below, the CPU time distribution strongly converges on the target proportional ratio. Over the 20-second execution window:



- **PID_A (30 Tickets):** Received the highest share of CPU ticks (approx. 553).
- **PID_B (20 Tickets):** Received the middle share of CPU ticks (approx. 367).
- **PID_C (10 Tickets):** Received the lowest share of CPU ticks (approx. 210).

While the variance inherent to a pseudo-random generator means the distribution is not perfectly rigid, Process A consistently received roughly 2.5x to 3x more CPU time than Process C. This successfully validates that the custom Lottery Scheduler is correctly calculating and executing proportional CPU allocation.